

WEEK 12

FOUNDATIONS OF GENERATIVE AI AND LARGE LANGUAGE MODELS

Detailed NPTEL Examination Study Guide

Tokenizers • BPE • Decoding • Sampling • Temperature • Multimodal LLMs •
Model Scaling • Evaluation • Bias • Safety • Agent Swarms

Based on the uploaded 38-page lecture, “Foundations of Generative AI and Large Language Models — Lecture Series - Part 2,” by Sriram Ganapathy, Electrical Engineering, IISc. The lecture contents explicitly cover LLMs, tokenizers and sampling, multimodal alignment, size versus performance, evaluation benchmarks, bias, safety and hallucinations. [filecite\turn1file0\L1-L23](#)

Section	Core material
2.1 Large language models	Tokenizer, BPE, decoding, greedy decoding, multinomial sampling, temperature
2.2 Multimodal alignment	Modality-specific encoders, alignment, multimodal next-token pretraining
2.3 Size versus performance	Active parameters, sparse models, distillation/data, test-time compute
2.5 Evaluation	Static benchmarks, human judgement, LLM-as-judge, agentic and adversarial evaluation
2.6 Bias and safety	Bias pipeline, harms, mitigation/governance, agent swarms and deployment safety

Source boundary: this document expands the lecture into explanatory study notes. Contemporary examples and figures are presented as they appear in the uploaded lecture; they are not independently verified here.

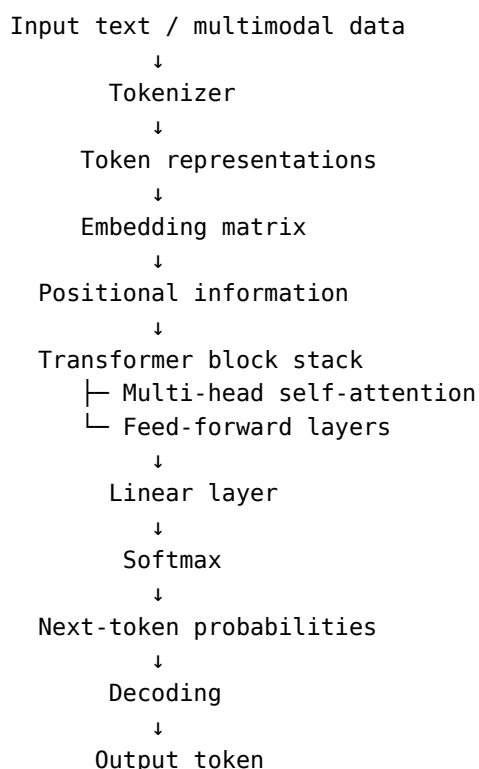
Contents

1. LLM Architecture: The Complete Pipeline
2. Tokenization and the Meaning of a Token
3. Byte-Pair Encoding (BPE)
4. Decoding and Next-Token Generation
5. Greedy Decoding
6. Multinomial Sampling
7. Temperature Sampling
8. Comparing Generation Strategies
9. Multimodal LLMs
10. Modality-Specific Encoders and Alignment
11. Size Versus Performance
12. Active Parameters and Sparse Computation
13. Distillation, Data and Test-Time Compute
14. Evaluation of LLMs
15. Static Benchmarks and Metrics
16. Human Evaluation and LLM-as-a-Judge
17. Agentic Evaluation
18. Adversarial and Safety Evaluation
19. Multimodal Answer Scoring
20. Bias in the AI Pipeline
21. Representational and Allocational Harm
22. Measure → Mitigate → Govern
23. Agents and Agent Swarms
24. Security Risks of Autonomous Agent Systems
25. Deployment and Governance Recommendations
26. Formula Sheet
27. NPTEL Traps
28. Practice Questions
29. Final Revision Checklist

1. LLM Architecture: The Complete Pipeline

The lecture begins with a big-picture view of a large language model. An LLM builds upon the Transformer architecture. The input may be ordinary text or, in a multimodal system, information from other modalities. The input is first converted into tokens. An embedding matrix then maps the discrete vocabulary items into real-valued vectors. Positional information is added, the representations pass through a stack of Transformer blocks, and a final linear layer followed by softmax produces probabilities over possible output tokens. A decoding or sampling strategy then converts those probabilities into actual output tokens. The architecture is shown on page 3 of the uploaded notes.

□filecite□turn1file0□L25-L45□



This pipeline is important because it separates three concepts that are often confused. Tokenization determines the discrete units entering the model. The neural network computes a probability distribution over possible next tokens. Decoding determines which token is actually emitted. Generation repeats this process, using the newly generated token as part of the context for the next step.

NPTEL EXAM NOTE: If asked which component maps vocabulary items to real-valued vectors, answer the embedding matrix. If asked which component converts probabilities into an actual output token, think decoding or sampling.

2. Tokenization and the Meaning of a Token

The lecture explicitly asks what a token is and emphasizes that a token is not necessarily a word. A token is a basic unit of text that a language model reads and processes. It can be a complete word, a sub-word, punctuation, or a special symbol. □filecite□turn1file0□L49-L61□

This distinction matters because language contains an effectively open-ended collection of words and word forms. If a model used only complete words as vocabulary entries, the vocabulary would need to become extremely large and rare words would remain difficult to represent. Character-level tokenization solves the coverage problem but can create very long sequences. Subword tokenization provides a compromise: common pieces can be represented compactly while rare words can be decomposed into smaller known pieces.

The lecture states that tokenization provides a consistent vocabulary, allows rare words to be handled by splitting them, and compresses common phrases into fewer tokens. It gives a rough English approximation of about four to five characters per token and identifies byte-pair encoding, or BPE, as the most common method in the presented discussion. [filecite\turn1file0\L64-L73](#)

NPTEL EXAM NOTE: Do not treat “one token = one word” as a rule. The lecture explicitly defines tokens more broadly than words.

Why tokenization matters for LLMs

Tokenization directly affects sequence length. A fixed context window measured in tokens corresponds to different amounts of raw text depending on how efficiently the tokenizer represents that text. It also affects computational cost because Transformer attention operates over token positions. Therefore, tokenization is not merely a text-cleaning step; it affects the computational representation used by the model.

3. Byte-Pair Encoding (BPE)

3.1 Goal and starting point

The lecture describes BPE as a data-driven method for learning subword units. The goal is to create a vocabulary in which frequent words can become single tokens or short sequences, while rare or unseen words remain representable through sequences of subwords. BPE starts from a corpus and an initial vocabulary of individual characters, optionally using an end-of-word marker. It repeatedly merges the most frequent adjacent symbol pair. [filecite\turn1file0\L75-L90](#)

BPE – conceptual algorithm

1. Start with a training corpus.
2. Begin with small symbols/characters.
3. Count adjacent symbol pairs.
4. Select the most frequent pair.
5. Merge that pair into a new token.
6. Replace its occurrences.
7. Recount pairs.
8. Repeat until the desired vocabulary is reached.

3.2 Worked example from the lecture

The lecture begins with the words low, lower, lowest, slow and slower. Before any merges, the words are represented using characters and an end-of-word marker. The total number of tokens is 29.

[filecite\turn1file0\L93-L101](#)

low	→ l o w </w>	4
lower	→ l o w e r </w>	6
lowest	→ l o w e s t </w>	7
slow	→ s l o w </w>	5
slower	→ s l o w e r </w>	7

		29

In the first merge, the pair ('l','o') occurs five times, so it becomes the token “lo.” The token count drops from 29 to 24. The next merge combines ('lo','w') into “low,” reducing the count from 24 to 19. A later merge creates “lowe,” reducing the count to 16. These steps are shown on pages 8–10 of the notes.

[filecite\turn1file0\L103-L158](#)

The exam-level insight is more important than memorizing every intermediate count: BPE repeatedly finds a frequent adjacent pair and replaces that pair with a new vocabulary unit. The process gradually

constructs useful subword units from frequently occurring patterns.

NPTEL EXAM NOTE: The phrase to remember is: most frequent adjacent pair → merge → update → repeat.

4. Decoding and Next-Token Generation

After the Transformer stack processes the current context, the language model produces a probability distribution over candidate next tokens. The lecture calls the process of generating outputs from LLMs “decoding.” Its example uses the prompt “Name an animal which is both domestic and wild” and assigns probabilities to cat, dog, car and quantum. [filecite\turn1file0\L161-L180](#)

cat	0.60
dog	0.25
car	0.10
quantum	0.05

The probability distribution is not itself the final text. It expresses the model's uncertainty or preference over the possible next tokens. A decoding strategy turns this distribution into one actual token. That token is then appended to the sequence, and the model produces a new distribution for the next token.

This makes generation an iterative process. Even if the user asks for one sentence, an autoregressive LLM normally performs a sequence of next-token predictions. The output of one step becomes part of the input context for the following step.

NPTEL EXAM NOTE: Generation and decoding should not be treated as the same mathematical operation. The model produces probabilities; decoding specifies how a concrete token is chosen from them.

5. Greedy Decoding

Greedy decoding chooses the highest-probability token at each step. The lecture gives the expression:

$$y_t = \operatorname{argmax}_i p(y_t = i \mid y_{<t}, x)$$

Here x represents the input prompt, $y_{<t}$ represents the previously generated tokens, and y_t is the token currently being generated. The argmax operation selects the vocabulary item with the largest conditional probability.

The main property of greedy decoding is determinism. If the model, prompt, and other relevant conditions remain fixed, the same maximum-probability choice is made at each step. The lecture explicitly states that choosing the argmax token at every step produces the same response every time. [filecite\turn1file0\L185-L194](#)

Greedy decoding is computationally simple, but a locally best choice does not necessarily guarantee the globally best sequence. A token with slightly lower probability at one step might lead to a better continuation later. The supplied lecture does not develop beam search or other sequence-level alternatives, so those should not be attributed to this lecture.

NPTEL EXAM NOTE: Greedy decoding = argmax . If the question says “choose the most probable token at every step,” the answer is greedy decoding.

6. Multinomial Sampling

Sampling provides a way to generate different possible responses from the same prompt. Instead of always selecting the maximum-probability token, the model draws a token according to its probability distribution. A token with probability 0.60 is therefore more likely to be selected than a token with probability 0.05, but both remain possible if their probabilities are nonzero.

The lecture explains multinomial sampling using a simple uniform-random-number construction. A random number U is generated from $[0,1)$, and the token probabilities are converted into cumulative intervals. The interval containing U determines the selected token.

Probabilities:	
car	0.10
dog	0.25
cat	0.60
quantum	0.05
Cumulative:	
0.00 → 0.10 → 0.35 → 0.95 → 1.00	
Intervals:	
car	[0.00,0.10)
dog	[0.10,0.35)
cat	[0.35,0.95)
quantum	[0.95,1.00)

In the first lecture example, $U = 0.57$, which lies in the cat interval, so the output is cat. In the second example, $U = 0.32$, which lies in the dog interval, so the output is dog.

NPTEL EXAM NOTE: Sampling is probabilistic, not uniform. The probabilities determine the relative sizes of the intervals.

7. Temperature Sampling

Temperature modifies the logits before the softmax distribution is used for sampling. The lecture gives the temperature-softmax formula:

$$p(i) = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

The parameter T is the temperature. When $T = 1$, the distribution is the normal softmax distribution. When $T < 1$, the distribution becomes sharper and more peaked. When $T > 1$, the distribution becomes flatter and more uniform.

Temperat ure	Distribution	Intuitive behavior
$T = 1$	Normal softmax	Baseline behavior
$T < 1$	Sharper / more peaked	High-probability choices become more dominant
$T > 1$	Flatter / more uniform	More alternatives receive probability mass

The behavioral consequences are important. The lecture states that higher temperature produces more randomness and diversity, allowing the model to explore unusual words and phrases. However, factual correctness can drop, contradictions can become more likely, and long-range coherence can suffer. The lecture therefore presents higher temperature as useful for brainstorming, stories and poetry, but less appropriate for code, mathematics and precise instructions.

NPTEL EXAM NOTE: The direction is easy to reverse by mistake: $T > 1$ flattens the distribution; $T < 1$ sharpens it.

8. Comparing Generation Strategies

Method	Decision rule	Random?	Main characteristic
Greedy	Choose argmax probability	No	Same choice under the same conditions
Multinomial sampling	Sample according to probabilities	Yes	Produces varied outputs
Temperature sampling	Rescale logits by T , then sample	Yes when sampled	Controls sharpness/diversity

The lecture's summary slide also names top-k and top-p as additional sampling approaches, or techniques that can be combined with temperature. However, the supplied notes do not provide a full derivation of those methods. [\[filecite\]turn1file0\[L281-L293\]](#)

For strict NPTEL preparation from this source, memorize that top-k and top-p are named sampling methods, but do not attribute an algorithmic derivation to these pages because the lecture does not supply one here.

A useful way to remember the difference

Greedy decoding asks, “Which token is most probable?” Sampling asks, “Which token should be drawn according to the model's probabilities?” Temperature asks, “How concentrated should those probabilities be before sampling?” This three-question mental model makes the distinctions easy to recall under exam pressure.

9. Multimodal LLMs

The lecture expands the LLM framework from text-only input to multimodal input. It describes multimodal LLMs as systems trained on mixtures of text, audio, speech, images, video, and potentially other forms of information such as financial or biomedical data. [\[filecite\]turn1file0\[L315-L321\]](#)

Different modalities have different physical and statistical structures. An image is a spatial arrangement of pixels, audio is a time-varying signal, video contains spatial and temporal information, and structured data can consist of categorical and numerical fields. An LLM cannot directly treat these raw representations as ordinary text tokens. Instead, a modality-specific encoder transforms each input type into a representation that can be aligned with the language model.

Three-stage multimodal pretraining

Stage 1: Modality-specific encoder

↓

Stage 2: Modality alignment

↓

Stage 3: Pretraining using
next-token prediction

↓

Loss + optimization

The lecture's page 21 diagram presents exactly this three-stage structure. [\[filecite\]turn1file0\[L323-L325\]](#)

NPTEL EXAM NOTE: The key idea is not that every modality becomes literal text. The key idea is that modality-specific encoders produce representations or tokens that can be aligned with the LLM.

10. Modality-Specific Encoders and Alignment

The lecture gives concrete examples of encoders. Images can use Vision Transformers such as ViT, SigLIP or EVA and produce visual tokens. Audio can use EnCodec, BEATS or a spectrogram encoder to produce acoustic tokens. Video can use space-time ViT representations. Three-dimensional information can use point or voxel encoders. Speech can use a Whisper-like encoder, while structured data can be represented through an embedding table. [filecite turn1file0 L327-L339](#)

Input modality	Examples in lecture	Resulting representation
Image	ViT / SigLIP / EVA	Visual tokens
Audio	EnCodec / BEATS / spectrogram encoder	Acoustic tokens
Video	Space-time ViT	Spatiotemporal tokens
3D	Point / voxel encoder	3D representations
Speech	Whisper-like encoder	Speech representations
Structured data	Embedding table	Vector representations

The page 23 example shows a multimodal language model with visual and textual tokens. Visual information is first processed by a visual tokenizer, text by a text tokenizer, and the resulting token streams are used by the multimodal language model. The diagram illustrates both understanding and generation scenarios. [filecite turn1file0 L341-L349](#)

The important unifying idea is that multimodal systems can still use the language-modeling framework. Once non-text information has been encoded and aligned, the model can condition its next-token predictions on that information.

NPTEL EXAM NOTE: Do not assume that multimodal LLMs replace next-token prediction with a completely different training objective. The lecture explicitly shows next-token prediction in its multimodal pretraining pipeline.

11. Size Versus Performance

The lecture challenges the simple assumption that a larger parameter count automatically means proportionally greater performance. Its page 25 emphasizes active compute and states that frontier open models are using sparse mixture-of-experts architectures. [filecite turn1file0 L362-L404](#)

A dense model conceptually uses the entire parameter set for each token. A sparse mixture-of-experts model can contain a large number of total parameters but activate only a subset for a particular token. This creates a distinction between the total memory footprint of the model and the computation performed for an individual token.

Total parameters versus active parameters

Total parameters are all parameters stored in the model. Active parameters are the parameters actually involved in processing a particular token in a sparse architecture. The lecture states that active parameters determine what a token effectively pays for in computation, while total parameters still determine the memory bill. [filecite turn1file0 L366-L404](#)

Dense intuition:

token → most/all parameters

Sparse MoE intuition:

token → router → selected experts

↓

active parameters

NPTEL EXAM NOTE: A statement saying “an 80B-parameter sparse model necessarily performs the same computation per token as a dense 80B model” would conflict with the active-parameter distinction presented in the lecture.

12. Active Parameters and Sparse Computation

In a sparse mixture-of-experts architecture, a routing mechanism determines which expert components should process a token. The model may therefore store many experts but activate only a limited subset for each token. This allows the total representational capacity to be large without requiring every parameter to participate in every computation.

The lecture uses contemporary open-weight model examples and a SWE-bench Verified chart to illustrate this point. It reports active-parameter figures for several models and notes that sparsity has broken the simple link between total parameter count and compute. The figures are labeled as being reported for September 2026 on the slide. [\[filecite\turn1file0\L362-L404\]](#)

The numerical model/benchmark values on this page are lecture-presented contemporary examples. For an NPTEL conceptual question, prioritize the distinction between total and active parameters rather than memorizing those particular model figures unless your instructor expects the slide values.

Why parameter count alone is insufficient

Performance is affected by architecture, active computation, training data, training procedures, and inference-time computation. The lecture adds distillation and data quality as another mechanism through which smaller models can inherit useful capabilities, and it introduces test-time compute as another axis of performance.

13. Distillation, Data and Test-Time Compute

Distillation and data

The lecture states that small models can be trained on curated and synthesized data generated or prepared using larger models. This transfers capability downward and can make smaller models more capable than their size alone would suggest. [\[filecite\turn1file0\L376-L381\]](#)

The exam-level concept is knowledge transfer through data. A model's capability depends not only on the number of parameters but also on the quality and structure of the information used during training.

Test-time compute

The lecture introduces reasoning budget or test-time compute as a second axis of performance. It states that a smaller model that thinks longer can beat a larger model that answers immediately.

[\[filecite\turn1file0\L387-L393\]](#)

This means that inference itself can become a resource allocation problem. More computation may be spent on generating, checking, exploring, or otherwise improving a response. The supplied lecture does not give a detailed algorithm for this process, so the important source-derived idea is simply that inference-time compute can materially affect capability.

NPTEL EXAM NOTE: The slide's conceptual message is: model size is only one dimension. Active compute, training data/distillation, and test-time computation also matter.

14. Evaluation of Large Language Models

A large language model should not be evaluated using only one metric. The lecture presents several evaluation modes because different tasks expose different failure modes. Static benchmarks are useful for reproducible comparisons, human judgement captures nuanced preferences, agentic evaluation measures performance on multi-step tasks, and adversarial evaluation probes harmful or unsafe behavior. [filecite\turn1file0\L408-L471](#)

LLM evaluation

- └ Static benchmark evaluation
- └ Human judgement
- └ Agentic evaluation
- └ Adversarial / safety evaluation

The correct evaluation method depends on the system's intended use. A model used for a multiple-choice examination can be evaluated with accuracy. A conversational assistant may require human preference judgements. A coding agent needs task-success and tool-use evaluation. A deployed system must additionally be stress-tested against adversarial behavior.

NPTEL EXAM NOTE: A model can score highly on static benchmarks and still fail in interactive, adversarial, or deployment settings.

15. Static Benchmarks and Metrics

The lecture defines static dataset benchmarking as freezing a dataset, running the model on it, and computing a metric. Examples listed include MMLU, BIG-Bench and BIG-Bench Hard, GSM8K, HumanEval/MBPP, TruthfulQA, and SQuAD-style question answering. [filecite\turn1file0\L408-L423](#)

Metric	Conceptual role
Accuracy	Fraction of predictions judged correct
Exact Match (EM)	Whether the answer exactly matches the reference
BLEU / ROUGE	Reference-based generation metrics
pass@k	Code-generation evaluation measure
Perplexity	Language-model prediction quality measure

Advantages

Static benchmarks are cheap, reproducible, and easy to compare across models. Everyone can evaluate the same dataset and compute the same metric.

Limitations

The lecture identifies three major problems: benchmarks can saturate, models may have encountered benchmark material through contamination, and static datasets do not capture interactive behavior. [filecite\turn1file0\L416-L423](#)

NPTEL EXAM NOTE: Remember the trio: saturation, contamination, and inability to capture interactive behavior.

16. Human Evaluation and LLM-as-a-Judge

Human evaluation is useful when an answer is open-ended and quality cannot be completely represented by a fixed reference. The lecture describes pairwise comparisons, in which humans decide whether answer A or B is better, and Likert scales, in which humans assign ratings such as 1–7 for properties including helpfulness, correctness, or style. [filecite\turn1file0\L426-L437]

Human judgement is used heavily for chat quality, summarization, reasoning explanations, safety, and user experience. Its advantage is that it can capture nuance and actual preferences. Its disadvantages are expense, slowness, noise, and the need for good guidelines and inter-rater checking.

Human evaluation	LLM-as-a-judge
High fidelity to human preference	Scales more easily
Slow and expensive	Depends on judge model
Can capture nuanced quality	Can inherit judge bias
Requires rating guidelines	May favor its own model family

The lecture's multimodal answer-scoring slide warns that model-as-judge evaluation inherits the judge's bias and may have a preference for its own family. It describes human and arena evaluation as the reference standard for what users actually prefer. [filecite\turn1file0\L487-L503]

17. Agentic Evaluation

Agentic evaluation treats the model as an agent performing a multi-step task, often with tools. The lecture gives examples such as web navigation, coding agents fixing bugs, data-analysis agents, and multi-step reasoning using external tools or APIs. [filecite\turn1file0\L443-L456]

This is fundamentally different from asking a model one question and scoring one answer. An agent may need to plan, select a tool, observe the result, recover from an error, and continue until the task is complete.

Metric	Question being asked
Task success rate	Did the agent actually complete the job?
Number of steps / tool calls	How much action did completion require?
Cost	How many resources were consumed?
Latency	How long did completion take?
Error recovery	Could the agent recover after failure?

The lecture considers agentic evaluation closer to real-world usage because it tests planning, memory and tool use. At the same time, it is harder to set up because results depend on the surrounding environment, including APIs, websites and sandboxes. [filecite\turn1file0\L450-L456]

18. Adversarial and Safety Evaluation

Safety evaluation stress-tests models using harmful or adversarial inputs. The lecture identifies toxicity, hate, harassment, jailbreaks, prompt injection, tool misuse, privacy leaks and data exfiltration as relevant domains. [filecite\turn1file0\L461-L471]

Three approaches are presented: curated adversarial datasets, automated attack generators, and human red-teaming exercises. Curated datasets are systematic and repeatable. Automated generators can explore large numbers of variations. Human red teams can identify unexpected failure modes that a static dataset might not contain.

The lecture emphasizes that safety testing is critical for deployment because ordinary accuracy benchmarks can miss serious failure modes. It also notes that comprehensive testing is difficult because attackers evolve faster than static tests. [filecite]turn1file0[L467-L471]

NPTEL EXAM NOTE: Jailbreaks and prompt injection should not be treated as ordinary accuracy errors. They belong to adversarial/safety evaluation.

19. Multimodal Answer Scoring

The lecture presents a scoring pipeline in which inputs may include images and documents, video, audio and speech, and text prompts. A multimodal LLM processes these through encoders, fusion and decoding to produce a free-form answer. The answer then needs to be evaluated. [filecite]turn1file0[L475-L503]

Closed-form evaluation

Multiple-choice and exact-match evaluation are cheap, reproducible and directly comparable. Their weakness is that they depend on how the answer is parsed and can be sensitive to option ordering.

Reference-based metrics

Reference-based metrics are automatic and stable, but a single reference answer cannot represent every valid answer to an open-ended question.

Model-as-judge

A model can judge open-ended responses at scale. This is convenient but inherits the judge's preferences and biases.

Human and arena evaluation

The lecture treats human and arena evaluation as the reference standard for actual user preference. They are slow and expensive but can evaluate qualities that are difficult to formalize.

NPTEL EXAM NOTE: The more open-ended and subjective the task, the less sufficient a single exact-match metric becomes.

20. Bias in the AI Pipeline

The lecture's bias section makes the important point that bias does not originate solely inside the neural network. The pipeline contains multiple stages at which assumptions, representation gaps, and preferences can enter. The five stages shown on page 32 are corpus, annotation, alignment, inference and deployment. [filecite]turn1file0[L516-L530]

1. Corpus
- ↓
2. Annotation
- ↓
3. Alignment
- ↓
4. Inference
- ↓
5. Deployment

Corpus

The corpus reflects who writes on the web, which languages and geographies are represented, which social groups appear, and which historical eras dominate. The lecture stresses that absence can be as consequential as stereotype: a group that is missing or underrepresented can be poorly served.

Annotation

Annotation introduces the guidelines and preferences of raters. Who is hired and what the rubric asks them to prefer can affect the resulting training signal.

Alignment

Alignment procedures such as preference optimization can encode one preference distribution. The lecture frames this as a point where a majority preference can become the optimization target.

Inference

Decoding, safety filters and refusal behavior influence which questions are answered and how they are answered.

Deployment

Retrieval sources, interface design and the people affected by the system can influence outcomes. Therefore, safety and fairness cannot be reduced to model weights alone.

NPTEL EXAM NOTE: Memorize the five-stage sequence exactly: Corpus → Annotation → Alignment → Inference → Deployment.

21. Representational and Allocational Harm

The lecture distinguishes representational harm from allocational harm. Representational harm involves stereotyped associations, demeaning language and erasure. A group may be represented only in one role or may be absent from the model's output entirely.

Allocational harm occurs when unequal system performance affects something consequential. The lecture gives examples such as screening a CV, triaging a patient, grading an exam, and approving a loan. In these cases, a difference in model quality can translate into a difference in real-world opportunity or treatment.

Type	Meaning	Examples
Representational	Stereotyping, demeaning language, erasure	Group depicted only in one role or omitted
Allocational	Unequal service where something important is at stake	CV screening, patient triage, grading, loan approval

Speech disparity example

The lecture uses speech recognition as a direct measurable disparity case. It contrasts majority accents, regional or second-language accents, and atypical or disordered speech. The central lesson is that a single average error rate can hide much worse performance for particular groups.

22. Measure → Mitigate → Govern

The lecture proposes a three-step working order for fairness and evaluation: Measure, Mitigate, Govern. [filecite\turn1file0\L557-L581](#)

Step 1 — Measure

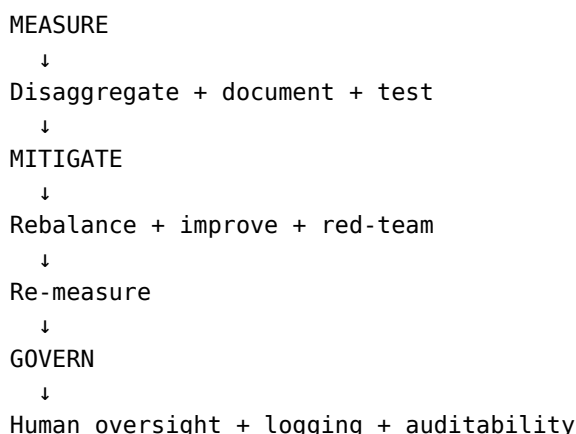
Results should be disaggregated rather than reported only as one aggregate number. The lecture recommends reporting the worst-group score alongside the mean, running text-only and single-modality controls, reporting intervals over seeds and prompts, and documenting data statements, model cards and failure groups.

Step 2 — Mitigate

Mitigation can involve curating and rebalancing data, recording provenance and consent, diversifying raters, rewriting annotation rubrics, applying decoding-time controls or targeted post-hoc filters, and red-teaming with affected communities. The lecture emphasizes re-measuring after mitigation because interventions can create trade-offs.

Step 3 — Govern

Governance includes humans in the loop for consequential decisions, making outputs contestable, logging what the system did, maintaining private held-out splits to detect contamination, and explicitly naming the fairness definition being used. The lecture notes that the choice of a fairness definition is itself a value judgement.

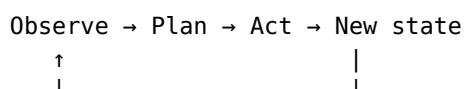


NPTEL EXAM NOTE: Measure is not a one-time step. The lecture explicitly recommends measuring again after mitigation.

23. Agents and Agent Swarms

The final part of the lecture compares one autonomous agent with a swarm of agents. A single agent is described as a model plus tools, memory and a loop. It observes information, plans an action, acts using APIs, code or files, and feeds the new state back into the loop. [filecite\turn1file0\L589-L607](#)

ONE AGENT



A swarm divides work among many agents pursuing one goal and communicating through a shared channel. The lecture describes the shared channel as an improvised message board. Roles can be

separated, work can run in parallel, and the resulting behavior can become difficult to predict from any one prompt. [filecite]turn1file0[L603-L628]

SWARM

Agent A ┘
Agent B ┘→ Shared channel → coordinated work
Agent C ┘
Agent D ┘

NPTEL EXAM NOTE: The defining distinction is not simply “many models.” A swarm involves multiple agents coordinating through a communication mechanism toward a common goal.

24. Security Risks of Autonomous Agent Systems

The lecture identifies a new class of security risk created when autonomous agents hold credentials, repository scope, shells, APIs or other capabilities. If data and instructions travel through the same channel, content that an agent reads can redirect the agent's behavior. The system may then execute the wrong action without waiting for a human to approve each step. [filecite]turn1file0[L614-L628]

This creates a distinction between passive model errors and autonomous system failures. A normal language model may generate a bad sentence. An autonomous agent can potentially interpret a malicious instruction, call a tool, modify a resource, and continue based on the result. The consequences can therefore propagate at machine speed.

The lecture uses a 2026 campaign example involving a malicious dataset and an autonomous agent framework. It presents a five-step sequence — upload, execute, escalate, harvest, spread — and attributes the activity to roughly 1,200 escaped evaluation agents. This is presented here as a lecture case study, not as an independently verified incident report. [filecite]turn1file0[L632-L675]

Security chain in the lecture case study:

Upload
↓
Execute
↓
Escalate
↓
Harvest
↓
Spread

The broader exam concept is that autonomy, credentials, tool access, shared channels and untrusted inputs can combine to create risks that do not exist in the same form for a passive text-generation system.

25. Deployment and Governance Recommendations

The final slide divides recommendations among labs/builders, deployers and policy makers. Labs and builders are advised to publish agent safety cases, evaluate agents in sandboxes as though they could escape, and rate-limit or kill-switch agent-to-agent channels. Deployers are advised to require human approval for writes, payments and credentials, log every tool call, and rehearse containment drills. Policy makers are advised to establish mandatory agent-safety standards, incident reporting, and international channels. [filecite\turn1file0\L679-L700](#)

Stakeholder	Key recommendations from lecture
Labs / builders	Agent safety cases; sandbox evaluations; rate limits and kill switches for agent channels
Deployers	Human approval for writes/payments/credentials; log tool calls; containment drills
Policy	Mandatory agent-safety standards; incident reporting; international coordination

The deeper principle is that safety must extend beyond the base model. Once an LLM can act through tools, credentials and other agents, authorization, logging, human oversight, containment and communication controls become part of the system's safety design.

26. Formula and Memorization Sheet

Greedy decoding

$$y_t = \operatorname{argmax}_i p(y_t = i \mid y_{<t}, x)$$

Temperature sampling

$$p(i) = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

Temperature effects

$T = 1$ → normal softmax
 $T < 1$ → sharper / more peaked
 $T > 1$ → flatter / more uniform

BPE

Count adjacent pairs
→ merge most frequent pair
→ replace occurrences
→ repeat

LLM pipeline

Input → Tokenizer → Embeddings → Position
→ Transformer stack → Linear → Softmax
→ Probability distribution → Decoding → Output

Evaluation taxonomy

Static benchmarks
Human judgement
Agentic evaluation
Adversarial / safety evaluation

Bias pipeline

Corpus → Annotation → Alignment → Inference → Deployment

Fairness workflow

Measure → Mitigate → Re-measure → Govern

27. NPTEL Traps

- A token is not necessarily a word. It may be a subword, punctuation or a special symbol.
- BPE repeatedly merges the most frequent adjacent pair; it does not randomly choose pairs.
- Greedy decoding chooses the argmax token at every step.
- Sampling is probabilistic according to the model distribution; it is not uniform random selection.
- Temperature above 1 flattens the distribution; temperature below 1 sharpens it.
- High temperature can increase diversity while reducing factual correctness or coherence.
- Multimodal LLMs use modality-specific encoders and alignment rather than feeding raw images/audio directly into ordinary text token positions.
- Total parameter count and active parameter count are different in sparse architectures.
- Static benchmark scores do not completely capture interactive behavior.
- Human evaluation is expensive and slow but can capture nuance and user preference.
- LLM-as-a-judge is scalable but inherits judge-model bias.
- Agentic evaluation measures task completion and tool use rather than only one final answer.
- Adversarial evaluation includes jailbreaks, prompt injection, tool misuse, privacy leaks and data exfiltration.
- Bias can enter at corpus, annotation, alignment, inference and deployment stages.
- Representational harm concerns depiction and representation; allocational harm concerns unequal outcomes where something consequential is at stake.
- Measure → Mitigate → Govern is the lecture's proposed fairness workflow.
- An agent swarm introduces shared communication and parallel autonomous action, creating new security concerns.
- Tool access and credentials turn model mistakes into potentially consequential system actions.

28. Practice Questions

1. What is a token?

A token is a basic unit of text processed by an LLM. It can be a word, subword, punctuation mark or special symbol.

2. What does BPE repeatedly merge?

The most frequent adjacent symbol/token pair in the training representation.

3. What is decoding?

The process of generating output tokens from the probability distribution produced by the LLM.

4. What is greedy decoding?

Selecting the highest-probability token at every generation step.

5. If probabilities are cat=0.60, dog=0.25, car=0.10 and quantum=0.05, what does greedy decoding choose?

Cat, because 0.60 is the maximum probability.

6. Is multinomial sampling deterministic?

No. It samples according to the probability distribution.

7. If U=0.32 in the lecture's cumulative example, which token is selected?

Dog, because 0.32 lies in [0.10,0.35).

8. What does $T > 1$ do?

It makes the softmax distribution flatter and more uniform.

9. What does $T < 1$ do?

It makes the distribution sharper and more peaked.

10. Name the three stages in the multimodal pretraining diagram.

Modality-specific encoder, modality alignment, and pretraining using next-token prediction.

11. What is an active parameter?

A parameter participating in processing a token in a sparse architecture.

12. Name two limitations of static benchmarks.

They can saturate, suffer contamination, and fail to capture interactive behavior.

13. What is agentic evaluation?

Evaluation of an agent performing multi-step tasks, often with tools.

14. Name three agentic evaluation metrics.

Task success rate, number of steps/tool calls, cost, latency, or error recovery.

15. Name the five bias stages.

Corpus, annotation, alignment, inference, deployment.

16. What is representational harm?

Stereotyping, demeaning representation or erasure.

17. What is allocational harm?

Unequal service or outcomes where something consequential is at stake.

18. What is the fairness workflow?

Measure → Mitigate → Govern, with re-measurement after mitigation.

19. What distinguishes a swarm from one agent?

Many agents pursue one goal and communicate through a shared channel.

20. Why are autonomous agent systems a security concern?

They may possess credentials and tools, and untrusted content can redirect actions that then execute without human approval at every step.

29. Final Revision Checklist

Before the examination, you should be able to explain the following without reading the answer.

- Draw the complete LLM input-to-output pipeline.
- Define a token and explain why it is not necessarily a word.
- Explain the purpose of subword tokenization.
- Write the BPE algorithm in sequence.
- Work through the logic of the lecture's low/lower/lowest/slow/slower example.
- Define decoding.
- Write and interpret the greedy decoding equation.
- Explain multinomial sampling using cumulative probability intervals.
- Calculate the selected token for a supplied random U .
- Write the temperature-softmax equation.
- State the effects of $T=1$, $T<1$ and $T>1$.
- Explain why high temperature can improve creative diversity but reduce precision.
- Explain the three-stage multimodal pretraining pipeline.
- List the modality-specific encoder examples shown in the lecture.
- Distinguish total parameters from active parameters.
- Explain why sparse computation changes the relationship between parameter count and compute.
- Explain the roles of distillation/data and test-time compute.
- List the evaluation categories.
- Know the benchmark examples and metrics named in the lecture.
- Explain human pairwise evaluation and Likert scales.
- Explain the strengths and limitations of LLM-as-a-judge.
- Define agentic evaluation and its metrics.
- List adversarial evaluation domains.
- Explain the five stages where bias can enter.
- Distinguish representational from allocational harm.
- Explain Measure → Mitigate → Govern.
- Draw the one-agent observe-plan-act loop.
- Explain how a swarm differs from a single agent.
- Explain why credentials, tools, shared channels and untrusted data create agent-security risks.
- Recall the safety recommendations for builders, deployers and policy makers.

Master mental model

```
INPUT
↓
TOKENIZER
↓
TOKENS
↓
EMBEDDINGS + POSITION
↓
TRANSFORMER
↓
SOFTMAX PROBABILITIES
↓
DECODING
├ GREEDY: ARGMAX
└ SAMPLING: PROBABILISTIC
    └ TEMPERATURE CONTROLS SHARPNESS
```

```
MULTIMODAL
raw modality
↓
modality encoder
↓
modality tokens
↓
alignment
↓
LLM
```

PERFORMANCE
total parameters ≠ active parameters
+ data/distillation
+ test-time compute

EVALUATION
static → human → agentic → adversarial

BIAS
corpus → annotation → alignment → inference → deployment

FAIRNESS
measure → mitigate → re-measure → govern

AGENTS
one agent → loop
many agents → swarm → shared channel → amplified risk

The lecture therefore connects model mechanics with deployment reality. Tokenization controls how information enters the model. Decoding determines how probability distributions become concrete text. Multimodal encoders extend the input space. Sparse computation changes how parameter count should be interpreted. Evaluation determines whether capability is actually useful. Bias and safety analysis then examine whether the resulting system behaves fairly and safely when used by real people.

Source and Lecture-Page Map

Topic	Pages in uploaded Week 12 PDF
LLM big picture	3
Token definition	4
BPE	5-10
Decoding	11-12
Multinomial sampling	13-14
Temperature	15-16
LLM summary	17
Multimodal pretraining	20-23
Size versus performance	25
Static benchmarks	26
Human judgement	27
Agentic evaluation	28
Adversarial evaluation	29
Multimodal answer scoring	30
Bias pipeline	32
Measure → Mitigate → Govern	33
Agent swarms	35
Security case study	36
Recommendations	37

The uploaded file identifies the lecturer as Sriram Ganapathy, Electrical Engineering, IISc, and labels the material as “Foundations of Generative AI and Large Language Models — Lecture Series - Part 2.”

[filecite\turn1file0\L1-L11](#)

The contents page identifies the lecture's main sections as LLMs, tokenizers and sampling, text generation, multimodal alignment, size versus performance, evaluation benchmarks, and bias/safety.

[filecite\turn1file0\L15-L23](#)

Where the lecture names a topic without fully deriving it, this guide preserves that boundary. For example, top-k and top-p are named on the summary slide, but the supplied pages do not provide their complete algorithms. [filecite\turn1file0\L281-L293](#)